

Textbook TTS + Figure Referencer — Ideation Document

Captured verbatim from session. Do not append.

Problem Statement

NotebookLM and similar tools rephrase good explanations from textbooks instead of preserving them. Simple TTS exists but has two gaps: (1) no awareness of diagrams/figures that require visual attention, and (2) no structural awareness of textbook content types. The goal is an Audible-style experience for dense learning material — specifically textbooks — where the listener is guided through content and told when to look at the screen.

Prototype Scope

A simple TTS + Figure Referencer.

Input: A PDF + a page range (e.g. pg10–20)

Output: An MP3 file containing TTS of text, with verbal prompts inserted to “look at Figure XYZ for this next bit.”

Processing Pipeline

Step 1: PDF Splice

Given a PDF and a page range, extract only the relevant pages.

Step 2: Content Classification — “Chunks”

Classify text sections into categories:

Category	Description

<code>info</code>	Regular paragraphs — the main explanatory content
<code>reference</code>	In-text citations, bibliography, footnotes
<code>exercise</code>	Question banks, problem sets, exercises
<code>illustrative_example</code>	Solved examples, worked problems
<i>(and more)</i>	Reserved for additional cases discovered during implementation

These are the **Chunks** — the primary units of TTS flow.

Step 3: Content Classification — “Figures”

Classify non-text elements into categories:

Category	Description
<code>referable</code>	Figures, images, diagrams, code examples that a Chunk refers to — requires user to look at screen
<code>table</code>	Tabular data
<code>misc_textbox</code>	Summary blurbs, callout boxes, “important points” sidebars — often visually distinct via formatting/borders/shading
<i>(and more)</i>	Reserved for additional cases

These are the **Figures** — elements that get referenced by Chunks, not read linearly.

Step 4: Chunk → Figure Mapping

Create a mapping from each Chunk to the Figures it references. This is the core intelligence of the pipeline.

Mapping strategy (proposed): Lightweight NLP or regex/pattern-matching to detect in-text figure references (e.g. “as shown in Figure 3.2”, “see Table 1”, “refer to the diagram above”). No complex semantic reasoning needed — explicit references are usually present.

Model constraints:

- Must run on at least a GTX 1650 Ti at reasonable speed

- No server infrastructure — personal laptop or free-tier cloud
- Lightweight is the priority

Candidate approaches:

- Lightweight local NLP (e.g. spaCy, regex over extracted text)
 - Claude Haiku API (outsourced classification/mapping — no local GPU requirement)
 - Hybrid: regex for explicit references, Haiku for ambiguous cases
-

Step 5: Sequence / Data Structure for TTS

Define how the Chunk→Figure mapping is passed to the TTS engine.

Proposed approach: An ordered sequence where each entry is a Chunk paired with a list of its referenced Figures. Before reading a Chunk, the TTS system reads out “Please refer to [Figure XYZ]” for each referable Figure in the list.

Open questions:

- Is a simple ordered list sufficient or does a more exotic DS (stack, priority queue, etc.) offer real benefit?
 - Should `misc_textbox` Figures be spoken, skipped, or offered as an aside?
 - Should `exercise` Chunks be read or skipped by default?
-

Step 6: TTS Output

Convert the ordered Chunk sequence (with injected Figure prompts) to an MP3.

TTS approach: Simple phrase injection before each Chunk — “Refer to Figure 3.2” — then read the Chunk text.

Open for later: Better audio experience ideas (e.g. earcon/audio cue instead of verbal prompt, distinct voice for prompts vs. content).

Key Design Constraints

- KISS principle: simplest thing that works first

- No server required for the prototype
 - GTX 1650 Ti is the floor for any local model
 - Personal GitHub / Vercel / free-tier cloud deployment target
 - Not a general-purpose tool — textbooks are the specific target
-

Open Questions / Future Work

- Handling multi-column PDF layouts (common in textbooks)
- Handling math equations (MathML, rendered images)
- Handling code blocks in CS textbooks
- Per-category TTS behaviour (skip exercises? summarise misc_textbox?)
- Better audio UX than raw verbal prompts
- Mobile companion app to display figures on cue